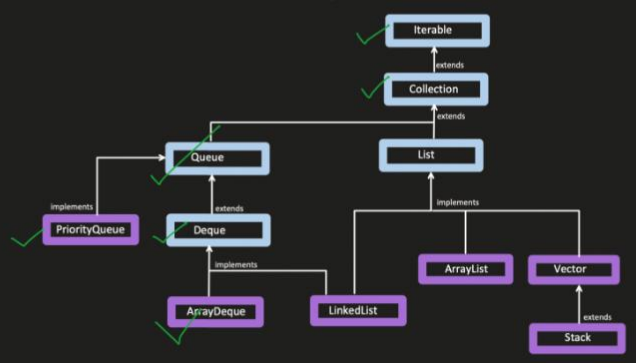
List:

List is a ordered collection of an objects. In which duplicate values can be stored.

How does it differs from Queue?

- Queue is also a collection of an objects but in queue insertion/removal/access can only happen either at start or end of the queue. While in List, data can be inserted, removed or access from anywhere.

- In List, user can decide where to insert or access using index (starting from 0)



Methods available in List Interface:

Methods available in Collection Interface + new Methods defined in List Interface

Collection Interface Methods as explained previously:

S.NO	METHODS	USAGE
1.	size()	It returns the total number of elements present in the collection.
2.	isEmpty()	Used to check if collection is empty or has some value. It return true/false.
3.	contains()	Used to search an element in the collection, returns true/false.
4.	toArray()	It convert collection into an Array.
5.	add(E element)	insert element at end of the collection
6.	remove(E element)	remove the first occurence of the element from the list
7.	addAll(Collection c)	Used to insert all the element of the specified collection in the end of the list.
8.	removeAll(Collection c)	Remove all the elements from the collections, which are present in the collection passed in the parameter.
9.	clear()	Remove all the elements from the collection.
10.	equals()	Used to check if 2 collections are equal or not.
11.	stream() and parallelStream()	Provide effective way to work with collection like filtering, processing data etc.
12.	iterator()	As Iterable interface added in java 1.5, so before this, this method was used to iterate the collection and still can be used.

New Methods defined in the List Interface:

S.No.	Methods Available in Queue Interface	Usage
1.	add(int index, E element)	- Insert the element at the specific position in the list. if there is any element present at that position, it shift it to its Right
2.	addAll(int index, Collection c)	Insert all the elements of the specified collection into this list at specific index, and shift the element at this index and subsequent element to the right.
3.	replaceAll(UniaryOperator op)	relaces each element of the list, with the resulf of appliying the operator the element.
2.	sort(Comparator c)	sort based on the order provided by the comparator.
3.	get(int index)	Return the element at the specified position in the list.
4.	set(int index, Element e)	Replace the element at the specified index in the list with the element provided.
5.	remove(int index)	Remove the element from the index and shift subsequent elements to the left.
6.	indexOf(Object o)	Returns the index of the first occurrence of the specified element in the list. -1 if list does not contains the element.
7.	lastIndexOf(Object o)	Returns the index of the lastoccurrence of the specified element in the list. -1 if list does not contains the element.
8.	ListIterator<E> listIterator()	listIterator return the object of ListIterator. ListIterator (interface) extends from the Iterator (interface) It has all the methods which are avaiable in **Iterator** and helps to iterate in forward direction like: Method Name: hasNext() — Usage: Returns true, if there are more elements in collection Method Name: next() — Usage: Returns the next element in the iteration Method Name: remove() — Usage: Removes the last element returned by iterator + New methods which are intorduced in ListIterator, which help to iterate in backward direction: Method Name: boolean hasPrevious() — Usage: Returns true, if there are more elements in list while traversing in reverse order. Else throw exception Method Name: E previous() — Usage: Returns the previous element and move the cursor position backward. Method Name: int nextIndex() — Usage: Returns the index of the next element. (return the size of the list, if its at the end of the list) Method Name: int previousIndex() — Usage: Returns the index of the previous element. (return -1 of the list, if its at the begining of the list) Method Name: set(E e) — Usage: Replaces the last element returned by next or previous with the specified element. Method Name: add(E e) — Usage: Insert the specified element immediately before the element that would be returned by the next and after the element that would be returned by the previous. Subsequent call to next() would be uaffected.
9.	ListIterator<E> listIterator(int index)	start the iterator from the specific index. specified index indicates the first element that would be returned by an initial call to next.
10.	List<E> subList(int fromIndex, int toIndex)	return the portion of the list. fromIndex - Inclusive toIndex - Exclusive if fromIndex == toIndex, return sublist is empty. Any changes in sublist, will change in main list also and vise versa.

```java
public class Main {
    public static void main(String[] args) {

        List<Integer> list1 = new ArrayList<>();

        //add(int index, Element e)
        list1.add( index: 0,  element: 100);
        list1.add( index: 1,  element: 200);
        list1.add( index: 2,  element: 300);

        //addAll(int index, Collection c)
        List<Integer> list2 = new ArrayList<>();
        list2.add( index: 0,  element: 400);
        list2.add( index: 1,  element: 500);
        list2.add( index: 2,  element: 600);

        list1.addAll( index: 2, list2);
        list1.forEach((Integer val) -> System.out.println(val));

        //replaceAll(UnaryOperator op)
        list1.replaceAll((Integer val) -> -1 * val);
        System.out.println("after replace all");
        list1.forEach((Integer val) -> System.out.println(val));


        //sort(Comparator c)
        list1.sort((Integer val1, Integer val2) -> val1 - val2);
        System.out.println("after sorting in increasing order");
        list1.forEach((Integer val) -> System.out.println(val));

        //get(int index)
        System.out.println("value present at index 2 is: " + list1.get(2));

        //set(int index, Element e)
        list1.set(2, -4000);
        System.out.println("after set method");

        list1.forEach((Integer val) -> System.out.println(val));

        //remove(int index)
        list1.remove( index: 2);
        System.out.println("after removing");
        list1.forEach((Integer val) -> System.out.println(val));

        //indexOf(Object o)
        System.out.println("index of -200 Integer object is: " + list1.indexOf(-200));


        //need to provide the index in listIterator, from where it has to start.
        ListIterator<Integer> listIterator1 = list1.listIterator(list1.size());

        //traversing backward direction
        while (listIterator1.hasPrevious()){
            int previousVal = listIterator1.previous();
            System.out.println("traversing backward: " + previousVal + " nextIndex: " + listIterator1.nextIndex() + " previous index: " + listIterator1.previousIndex());
            if(previousVal == -100) {
                listIterator1.set(-50);
            }
        }
        list1.forEach((Integer val) -> System.out.println("after set:" + val));

        //traversing forward direction
        ListIterator<Integer> listIterator2 = list1.listIterator();
        while (listIterator2.hasNext()){
            int val = listIterator2.next();
            System.out.println("traversing forward: " + val + " nextIndex: " + listIterator2.nextIndex() + " previous index: " + listIterator2.previousIndex());
            if(val == -200) {
                listIterator2.add(-100);
            }
        }
        list1.forEach((Integer val) -> System.out.println("after add:" + val));

        List<Integer> subList = list1.subList(1, 4);
        subList.forEach((Integer val) -> System.out.println("sublist: " + val));

        subList.add(-900);
        list1.forEach((Integer val) -> System.out.println("after value added in sublist: " + val));
    }
}
```

```
Output:
100
200
400
500
600
300
after replace all
-100
-200
-400
-500
-600
-300
after sorting in increasing order
-600
-500
-400
-300
-200
-100
value present at index 2 is: -400
after set method
-600
-500
-4000
-300
-200
-100
after removing
-600
-500
-300
-200
-100
index of -200 Integer object is: 3
traversing backward: -100 nextIndex: 4 previous index: 3
traversing backward: -200 nextIndex: 3 previous index: 2
traversing backward: -300 nextIndex: 2 previous index: 1
traversing backward: -500 nextIndex: 1 previous index: 0
traversing backward: -600 nextIndex: 0 previous index: -1
after set:-600
after set:-500
after set:-300
after set:-200
after set:-50
traversing forward: -600 nextIndex: 1 previous index: 0
traversing forward: -500 nextIndex: 2 previous index: 1
traversing forward: -300 nextIndex: 3 previous index: 2
traversing forward: -200 nextIndex: 4 previous index: 3
traversing forward: -50 nextIndex: 6 previous index: 5
after add:-600
after add:-500
after add:-300
after add:-200
after add:-100
after add:-50
sublist: -500
sublist: -300
sublist: -200
after value added in sublist: -600
after value added in sublist: -500
after value added in sublist: -300
after value added in sublist: -200
after value added in sublist: -900
after value added in sublist: -100
after value added in sublist: -50
```

Time Complexity:

- Insertion:
  O(1) : when inserting the element at the end of the array. And space is sufficient
  O(n) : when inserting the lement at the particular index of the array, then it require shifting of the values,
  O(n) : when array size threshold reached and try to insert an element at end, then also its O(n) as values are copied to new array with bigger array size.

- Deletion: O(n), we have to shift all the values

- Search: O(1)

Space Complexity:
O(n)

Collection Till Now	isThreadSafe	Maintains Insertion Order	Null Elements allowed	Duplicate elements allowed	Thread safeVersion

ArrayList	No	Yes	Yes	Yes	CopyOnWriteArrayList
					`public class Test {`
					`public static void main(String args[]){`
					`List<Integer> list = new CopyOnWriteArrayList<>();`
					`list.add( index: 0, element: 100);`
					`System.out.println(list.get(0));`
					`}`
					`}`

LinkedList: Data structure used is Linked List

- **LinkedList implements both Deque and List interface.**
- Means it support Dequeue methods like : "getFirst", "getLast", "removeFirst", "removeLast" etc...

+

It also support index based operations like List : "get(index)" , "add(index, object)" etc.

```java
public class LinkedListExample {

    public static void main(String args[]){

        LinkedList<Integer> list = new LinkedList<>();

        //using deque functionality
        list.addLast( e: 200);
        list.addLast( e: 300);
        list.addLast( e: 400);
        list.addFirst( e: 100);
        System.out.println(list.getFirst());

        //using list functionality
        LinkedList<Integer> list2 = new LinkedList<>();
        list2.add( index: 0,  element: 100);
        list2.add( index: 1,  element: 300);
        list2.add( index: 2,  element: 400);
        list2.add( index: 1,  element: 200);
        System.out.println(list2.get(1) + " and " + list2.get(2));
    }
}
```

Output:

```
100
200 and 300
```

Time Complexity:

- Insertion at start and end : O(1)
- Insertion at particular index : O(n) for lookup of the index + O(1) for adding
- Search: O(n)
- Deletion at start or end: O(1)
- Deletion at specific index: O(n) for the lookup of the index + O(1) for removal

Space Complexity:
O(n)

Collection Till Now	isThreadSafe	Maintains Insertion Order	Null Elements allowed	Duplicate elements allowed	Thread safeVersion

LinkedList	No	Yes	Yes	Yes	CopyOnWriteArrayList
					`public class Test {`
					`public static void main(String args[]){`
					`List<Integer> list = new CopyOnWriteArrayList<>();`
					`list.add( index: 0,  element: 100);`
					`System.out.println(list.get(0));`
					`}`
					`}`

Vector:

- **Exactly same as arrayList, elements can be access via index.**
- **But its Thread safe**
- Puts lock when operation is performed on vector.
- Less efficient then ArrayList as for each operation it do lock/unlock internally.

```java
public class VectorExample {

    public static void main(String args[]){

        Vector<Integer> obj = new Vector<>();
        obj.add( index: 0,  element: 200);

        System.out.println(obj.get(0));
    }
}
```

Collection Till Now	isThreadSafe	Maintains Insertion Order	Null Elements allowed	Duplicate elements allowed	Thread safeVersion
Vector	Yes	Yes	Yes	Yes	N/A

Stack:

- **Represent LIFO (Last in First out) operation**
- **Since it extends Vector, its method is also Synchronized.**

How its different from Deque : Deque is not thread safe, stack is.

```java
public class StackExample {

    public static void main(String args[]){

        Stack<Integer> stack = new Stack<>();
        stack.push( item: 1);
        stack.push( item: 2);

        System.out.println(stack.pop());

    }
}
```

Time Complexity:

- Insertion : O(1)
- Deletion : O(1)
- Search: O(n)

Space Complexity:
O(n)

Collection Till Now	isThreadSafe	Maintains Insertion Order	Null Elements allowed	Duplicate elements allowed	Thread safeVersion
Stack	Yes	NO	Yes	Yes	N/A